120-Day Detailed DSA Roadmap

Python • Interview Preparation • Pattern Recognition • Built for a Full-Time Working Schedule

Primary goal: Build the ability to understand a problem, identify its pattern, explain the approach, and write a clean Python solution without depending on memorized answers.

Target: Approximately 100–110 quality problems, with heavy revision and repetition.

1. How to Use This Plan

Weekdays: 30–45 minutes of DSA. On stronger days, extend to 60 minutes.

Weekend: 1.5–2.5 hours for revision, deeper theory and harder problems.

Do not solve problems randomly. Each problem belongs to a pattern. Learn the pattern first, then solve variations.

Suggested session structure: 5–10 min review → 20–30 min problem attempt → 10–15 min solution review/debugging → 5 min notes.

Rule: Spend approximately 20–25 minutes genuinely attempting an Easy problem and 30–40 minutes on an appropriate Medium. If stuck, study the key idea, close the solution, and re-code it yourself.

2. The Four-Step Problem-Solving Process

Step 1 — Understand: What is the input? What must be returned? What are the constraints? Work through a small example manually.

Step 2 — Brute force: Write or describe the simplest correct approach. Estimate time and space complexity.

Step 3 — Pattern: Ask whether the problem involves hashing, two pointers, sliding window, stack, binary search, tree traversal, graph traversal, recursion or DP.

Step 4 — Code: Write the solution in Python, dry-run it, test edge cases, then explain complexity.

Never immediately watch a solution. The struggle is where pattern recognition develops.

3. Revision System

24-hour review: Revisit difficult problems the next day.

7-day review: Re-solve without looking at notes.

30-day review: Solve again under light time pressure.

Wrong-answer log: For every important mistake, record the pattern, what confused you, the key insight, and the mistake you made.

Pattern notebook: Keep one page per pattern with when to use it, the core template, common mistakes and 3–5 representative problems.

4. Python Tools You Must Know for DSA

Lists: append, pop, indexing, slicing, sorting.

Dict: get, membership, items, keys, values.

Set: membership, add, remove/discard.

collections: defaultdict, Counter, deque.

heapq: heappush, heappop, heapify.

Useful patterns: enumerate, zip, sorted(key=...), range, reversed.

Do not overuse clever Python shortcuts. Interview code should be readable and explainable.

DAYS 1–14 — ARRAYS, STRINGS AND HASHING

Goal: Become comfortable manipulating sequences and using dictionaries/sets for lookup and frequency counting.

Theory: indexing, traversal, nested loops, frequency arrays/maps, membership lookup and complexity of common operations.

Pattern question: Do I need fast lookup? Do I need to count occurrences? Am I repeatedly searching for something?

Days 1–3 — Array Fundamentals

Learn traversal, max/min, running totals and in-place modification.

Representative practice: Contains Duplicate, Maximum Subarray concept, Best Time to Buy and Sell Stock.

Focus on: one-pass scans, maintaining a running answer and avoiding unnecessary nested loops.

Days 4–6 — Strings

Learn indexing, slicing, two-direction traversal and character frequency.

Representative practice: Valid Palindrome, Valid Anagram, Reverse String.

Focus on immutable strings and when building a list is more efficient than repeated concatenation.

Days 7–10 — Hashmaps and Sets

Learn frequency maps, complement lookup and membership tests.

Representative practice: Two Sum, Valid Anagram, Group Anagrams, Longest Consecutive Sequence.

Key insight: trade extra space for faster lookup.

Days 11–14 — Consolidation

Re-solve at least 5 earlier problems without notes.

Do 3–4 mixed problems and classify the pattern before coding.

Checkpoint: You should know when a dictionary is better than repeated scanning.

DAYS 15–28 — TWO POINTERS, SLIDING WINDOW, PREFIX SUM

Goal: Learn to reduce repeated work by maintaining state while traversing an array or string.

Days 15–18 — Two Pointers

Learn opposite-direction pointers and same-direction pointers.

Representative practice: Valid Palindrome, Two Sum II, Container With Most Water, 3Sum.

Ask: Is the data ordered? Can two indices move toward a solution instead of checking every pair?

Common mistake: moving the wrong pointer without understanding why.

Days 19–23 — Sliding Window

Learn fixed-size and variable-size windows.

Representative practice: Maximum Average Subarray I, Longest Substring Without Repeating Characters, Minimum Size Subarray Sum.

Core idea: expand right pointer; shrink left pointer only when the window violates or exceeds a condition.

Write down the invariant: What exactly must remain true about the current window?

Days 24–26 — Prefix Sums

Learn running prefix totals and range queries.

Representative practice: Range Sum Query concept, Subarray Sum Equals K.

Key insight: store previously seen prefix sums when you need to count or find subarrays.

Days 27–28 — Review

Mixed set of 5–7 problems from arrays, hashing, two pointers and sliding window.

Checkpoint: Identify the pattern before writing code.

DAYS 29–38 — STACKS AND LINKED LISTS

Goal: Understand LIFO behavior and pointer manipulation.

Days 29–32 — Stack

Learn LIFO, push/pop, monotonic-stack intuition and matching problems.

Representative practice: Valid Parentheses, Min Stack, Daily Temperatures.

Ask: Does the problem require remembering the most recent unresolved item?

Days 33–38 — Linked Lists

Learn Node, next pointer, traversal and pointer reassignment.

Representative practice: Reverse Linked List, Merge Two Sorted Lists, Linked List Cycle, Middle of the Linked List.

Use diagrams. Before changing pointers, write down what each pointer currently references.

Common mistake: losing the remainder of the list because next was not saved before reassignment.

DAYS 39–46 — BINARY SEARCH AND RECURSION

Goal: Become comfortable with boundaries and recursive thinking.

Days 39–42 — Binary Search

Learn sorted-search boundaries, mid calculation and loop invariants.

Representative practice: Binary Search, Search Insert Position, Find Minimum in Rotated Sorted Array, Search in Rotated Sorted Array.

Ask: Is the search space ordered or can the answer space be searched monotonically?

Common mistakes: infinite loops, incorrect left/right updates and returning the wrong boundary.

Days 43–46 — Recursion

Learn base case, recursive case and call-stack reasoning.

Practice simple recursion before backtracking.

Representative practice: factorial-style exercises, recursive tree traversal and basic subsets intuition.

Checkpoint: Be able to trace recursive calls manually.

DAYS 47–60 — TREES AND HEAPS

Goal: Learn hierarchical traversal and priority-based selection.

Days 47–54 — Binary Trees

Learn TreeNode, DFS and BFS.

DFS traversals: preorder, inorder and postorder.

Representative practice: Maximum Depth of Binary Tree, Same Tree, Invert Binary Tree, Diameter of Binary Tree, Binary Tree Level Order Traversal.

Ask: Do I need depth-first recursion, or level-by-level BFS using deque?

Days 55–57 — BST

Learn BST ordering property.

Representative practice: Validate Binary Search Tree, Lowest Common Ancestor of a BST.

Understand why inorder traversal of a valid BST produces sorted values.

Days 58–60 — Heaps / Priority Queues

Learn min-heap behavior and Python heapq.

Representative practice: Kth Largest Element, Top K Frequent Elements.

Ask: Do I repeatedly need the smallest/largest item without sorting everything?

DAYS 61–72 — GRAPHS

Goal: Learn graph representation and traversal.

Days 61–63 — Graph Representation

Learn adjacency list, adjacency matrix concept and visited sets.

Understand directed vs undirected and weighted vs unweighted graphs.

Days 64–68 — BFS and DFS

Representative practice: Number of Islands, Flood Fill, Clone Graph.

DFS: explore deeply; BFS: explore level by level.

Always track visited nodes when cycles are possible.

Days 69–72 — Graph Patterns

Learn connected components, shortest-path intuition for unweighted graphs and grid traversal.

Representative practice: Rotting Oranges, Course Schedule concept.

Checkpoint: Know when to use recursion, iterative stack or deque.

DAYS 73–82 — BACKTRACKING

Goal: Learn choice → recurse → undo.

Days 73–76 — Core Backtracking

Learn decision trees, choices and backtracking state.

Representative practice: Subsets, Permutations.

Template: choose → recurse → undo.

Days 77–82 — Constraint Problems

Representative practice: Combination Sum, Word Search, N-Queens concept.

Focus on pruning and understanding state rather than memorizing templates.

DAYS 83–92 — BASIC DYNAMIC PROGRAMMING

Goal: Learn DP as repeated subproblems plus stored results.

Do not attempt advanced DP patterns yet.

Days 83–86 — 1D DP

Learn memoization and tabulation.

Representative practice: Climbing Stairs, House Robber.

Ask: What changes from one state to the next? What previous answers are reused?

Days 87–92 — DP Consolidation

Representative practice: Coin Change concept and one additional DP problem only after understanding states.

Write the state definition before coding.

Example question: What does dp[i] actually mean?

DAYS 93–105 — MIXED PRACTICE AND MEDIUM PROBLEMS

Goal: Transition from topic-based practice to interview-style recognition.

Create mixed sessions: one familiar pattern, one medium-level problem and one review problem.

Focus on weak areas identified in your mistake log.

Practice explaining your solution before coding.

DAYS 106–120 — INTERVIEW MODE

Goal: Retrieval, speed and communication.

Do not learn many new topics here.

Revisit your strongest representative problems from every pattern.

Use timed practice: Easy 15–20 min; Medium 30–40 min as a rough training target.

After each session, explain the brute force, optimized approach, complexity and edge cases.

Run 3–5 mock-style sessions during the final phase.

5. Recommended Problem Distribution

Arrays / Strings / Hashing: 25–30.

Two Pointers / Sliding Window / Prefix Sum: 15–20.

Stack / Queue: 8–10.

Linked Lists: 8–10.

Binary Search: 8–10.

Trees / Heaps: 15–20.

Graphs: 8–10.

Recursion / Backtracking: 5–8.

Basic DP: 5–8.

Total: approximately 100–110 quality problems, with many revisits.

6. Weekly Tracking Template

For each week, track: Days studied, problems attempted, problems solved independently, problems reviewed, patterns learned and recurring mistakes.

Example recurring mistakes: missed edge case, incorrect complexity, wrong pointer movement, hashmap initialization error, off-by-one boundary, recursion base case, visited-set omission.

Do not judge progress only by solved count. Independent solves and successful revisits matter more.

7. What to Do When You Get Stuck

0–10 minutes: understand examples and constraints.

10–25 minutes: try brute force and search for a pattern.

After that: read only enough explanation to understand the key insight.

Close the explanation and implement from memory.

If you cannot implement it, revisit the idea and try again.

Re-solve the problem the next day or within a week.

8. Final DSA Interview Checklist

Can you clarify the problem before coding?

Can you state the brute-force approach?

Can you identify the pattern?

Can you explain why the optimized approach works?

Can you code it cleanly in Python?

Can you test edge cases?

Can you state time and space complexity?

Can you communicate your reasoning while coding?

Final Mindset

The objective is not to finish a giant question list.

The objective is to see a problem and recognize: “This looks like a sliding window problem,” or “This needs hashing plus prefix sums,” and then construct the solution yourself.

Consistency beats intensity. With a full-time job, 30–45 focused minutes on most weekdays plus deeper weekend sessions is enough to build strong momentum over 120 days.